

NEUROLEARN

Sistema Operacional de Aprendizagem

Documentação Técnica

Arquitetura, stack, padrões e engenharia

Versão 2.0 · 8 de junho de 2026

NeuroLearn — Documentação Técnica

Versão: 2.1
Última atualização: 2026-06-08
Público: Engenheiros, contribuidores, IA coding agents

Visão Arquitetural

O NeuroLearn é uma aplicação web full-stack com arquitetura server-first (Next.js App Router), backend gerenciado (Supabase) e uma camada de domínio cognitivo pura em TypeScript.



% % % %
% % % %
% % % %
% % % %
% % % %
% % % %
% % % %

Stack Tecnológica

Tecnologia	Versão	Papel
Next.js	15.3.3	Framework full-stack (App Router)



Backend / Infraestrutura

Tecnologia	Versão	Papel
Supabase	latest	Auth + PostgreSQL + Edge Functions



Testes

Tecnologia	Versão	Papel
Vitest	4.1.8	Testes unitários

Observabilidade

Tecnologia	Versão	Papel
@sentry/nextjs	10.x	Error tracking, session replay, performance tracing

Utilitários

Tecnologia	Papel
docx	Geração de documentos Word

Estrutura de Pastas

```

% % % src/
% % % % app/ # Rotas Next.js (App Router)
% % % % % (app)/ # Grupo de rotas autenticadas
% % % % % % layout.tsx # Verifica sessão + carrega AppContext
% % % % % % % dashboard/
% % % % % % % library/
% % % % % % % focus/
% % % % % % % [contentId]/ # Rota dinâmica para sessão de foco
% % % % % % % review/
% % % % % % % active/
% % % % % % % skills/
% % % % % % % help/
% % % % % % % auth/
% % % % % % % login/page.tsx # Magic Link login
% % % % % % % signup/page.tsx # Cadastro com nome + email
% % % % % % % callback/route.ts # Handler OAuth / Magic Link
% % % % % % % api/
% % % % % % % ai/ # Endpoints de IA
% % % % % % % user/
% % % % % % % % delete/ # DELETE para LGPD
% % % % % % % layout.tsx # Root layout (fontes, tema, LGPD banner)
% % % % % components/
% % % % % % ui/ # Componentes base reutilizáveis
% % % % % % % Button.tsx
% % % % % % % Card.tsx
% % % % % % % Input.tsx
% % % % % % % Textarea.tsx
% % % % % % % FormField.tsx # Wrapper de campo com label + hint + erro
% % % % % % % FormError.tsx # Mensagem de erro com role="alert"
% % % % % % % FormHint.tsx # Texto auxiliar abaixo do campo
% % % % % % % LoadingButton.tsx # Botão com spinner durante submissão
% % % % % % % Toast.tsx # Notificação temporária
% % % % % % % Badge.tsx
% % % % % % % ProgressBar.tsx
% % % % % % % layout/
% % % % % % % Sidebar.tsx # Navegação lateral
% % % % % % % ThemeToggle.tsx # Toggle dark/light
% % % % % modules/ # Views por feature (lógica de página)
% % % % % % dashboard/DashboardView.tsx
% % % % % % library/
% % % % % % % LibraryView.tsx
% % % % % % % AddContentModal.tsx
% % % % % % % focus/
% % % % % % % review/ReviewView.tsx
% % % % % % % active/ActiveView.tsx
% % % % % % % skills/SkillsView.tsx
% % % % % % % help/HelpView.tsx
% % % % % % % engine/ # Cognitive Engine (algoritmos puros)
% % % % % % % index.ts # API pública
% % % % % % % spaced-repetition/ # SM-2 + scheduling
% % % % % % % retention/ # Ebbinghaus + forgetting risk
% % % % % % % mastery/ # Mastery score
% % % % % % % skill-evolution/ # Skill progression
% % % % % % % active-learning/ # Pirâmide de Glasser
% % % % % % % cognitive-score/ # Score global
% % % % % % % services/ # Acesso ao Supabase por domínio
% % % % % % % contentsService.ts
% % % % % % % flashcardsService.ts
% % % % % % % skillsService.ts
% % % % % % % sessionsService.ts
% % % % % % % reviewService.ts
% % % % % % % retentionService.ts
% % % % % % % cognitiveEventsService.ts
% % % % % % % localStorageService.ts
% % % % % % % store/
% % % % % % % AppContext.tsx # Estado global + reducer + sync Supabase
% % % % % % % ToastContext.tsx # Toasts (máx 3 simultâneos)
% % % % % % % FocusSessionContext.tsx # Estado global do timer (isRunning)
% % % % % % % components/
% % % % % % % % analytics/
% % % % % % % % % PostHogProvider.tsx # Init PostHog + page views + LGPD opt-in/out
% % % % % % % % % AnalyticsIdentifier.tsx # Identifica usuário no PostHog e Sentry
% % % % % % % % lgpd/
% % % % % % % % % ConsentBanner.tsx # Banner LGPD (nl_lgpd_consent)
% % % % % % % hooks/
% % % % % % % % useAppData.ts
% % % % % % % % useToast.ts
% % % % % % % % useTheme.ts
% % % % % % % % usePushNotifications.ts # Permissão reativa, subscribe/unsubscribe (PWA-01)
% % % % % % % % useAnalytics.ts # Hook type-safe: track(), identifyUser(), resetUser()
% % % % % % % lib/
% % % % % % % % supabase/
% % % % % % % % % client.ts # createBrowserClient
% % % % % % % % % server.ts # createServerClient (cookies)
% % % % % % % % % validation/
% % % % % % % % % schemas.ts # Schemas Zod centralizados

```

```

% % % security/
% % % rateLimit.ts
% % % validation.ts
% % % sanitize.ts
% % % rbac.ts
% % % logger.ts
% % % ai/
% % % client.ts
% % % prompts.ts
% % % types/
% % % index.ts # Tipos de domínio
% % % database.types.ts # Gerado pelo Supabase
% % % test/
% % % setup.ts # @testing-library/jest-dom
% % % tests/
% % % e2e/
% % % global.setup.ts # Auth setup via Supabase Admin API
% % % pages/ # Page Objects
% % % LibraryPage.ts
% % % ReviewPage.ts
% % % utils/helpers.ts
% % % auth.spec.ts
% % % app.spec.ts
% % % ux-01-validation.spec.ts
% % % library.spec.ts
% % % review.spec.ts
% % % accessibility.spec.ts
% % % docs/ # Documentação oficial
% % % public/
% % % landing.html
% % % middleware.ts # Rate limit + RBAC + session guard
% % % next.config.ts # Security headers + withSentryConfig + CSP expandida
% % % sentry.server.config.ts # Sentry init – server runtime
% % % sentry.edge.config.ts # Sentry init – edge runtime
% % % src/instrumentation.ts # Next.js hook: register() + onRequestError
% % % src/instrumentation-client.ts # Sentry init – browser (Session Replay)
% % % vitest.config.ts
% % % playwright.config.ts
% % % .env.local # Nunca commitar

```

Observabilidade

Sentry v10 — Error Tracking

O Sentry captura automaticamente erros em todos os runtimes:

Runtime	Arquivo de inicialização
Browser	`src/instrumentation-client.ts`

- Session Replay: gravações de sessão com maskAllText e blockAllMedia — 100% em erros, 10% em sessões normais
- onRequestError hook: captura erros em Server Components e Route Handlers
- global-error.tsx: tela de fallback PT-BR para erros de renderização React; reporta ao Sentry via captureException
- Source maps: upload automático no build via SENTRY_AUTH_TOKEN

PostHog — Product Analytics

O PostHog rastreia comportamento de usuário respeitando o consentimento LGPD:

- LGPD: accepted !' opt_in_capturing() | minimal/ausente !' opt_out_capturing()
- Page views: rastreados manualmente via usePathname + useSearchParams (Next.js App Router)
- Identificação: AnalyticsIdentifier chama posthog.identify(userId) ao entrar na área autenticada

Hook `useAnalytics`

```
const { track, identifyUser, resetUser } = useAnalytics()

// Eventos tipados — TypeScript garante propriedades corretas por evento
track('session_started', { content_id, phase, duration_secs })
track('achievement_unlocked', { achievement_id, xp_gained })
```

Variáveis de Ambiente de Observabilidade

Variável	Escopo	Obrigatória
<code>NEXT_PUBLIC_SENTRY_DSN</code>	Público	Para error tracking



Todas as vars devem ser adicionadas também no Vercel (Settings !' Environment Variables).

Padrões Arquiteturais

Server Components vs Client Components

- Server Components (padrão no App Router): layouts, páginas de carregamento, leitura de sessão
- Client Components ("use client"): qualquer componente com estado, eventos, hooks, contexto
- Regra: preferir Server Components; usar Client apenas onde necessário

Formulários

Todos os formulários seguem o padrão:



- AppContext: estado global da aplicação (contents, flashcards, skills, user)
 - Reducer puro exportado (`appReducer` + `EMPTY_STATE`)
 - Side effects assíncronos para sync com Supabase
- ToastContext: fila de toasts (máximo 3 simultâneos, LIFO)
- Sem Redux ou Zustand — React Context é suficiente para o escopo atual

Frontend Architecture

Roteamento



- useTheme() hook expõe theme e toggleTheme()

Componentes UI Base

Todos em src/components/ui/:

Componente	Props principais	Uso
<code>`FormField`</code>	<code>`label, required, hint, children`</code>	Wrapper de campo de formulário

Backend Architecture

Next.js API Routes

Localizadas em src/app/api/:

Rota	Método	Descrição
<code>`/api/user/delete`</code>	DELETE	Exclusão de dados do usuário (LGPD)

Middleware (`middleware.ts``)

Executado em todas as rotas (exceto `_next/static`, imagens, favicon):

1. Session guard: rota de app sem sessão !' redirect `/auth/login`

- 2. Auth redirect: rota de auth com sessão !' redirect /dashboard
- 3. Rate limiting: 5 req/15min por IP em /auth/*
- 4. RBAC: bloqueia /api/admin/* para roles insuficientes
- 5. Security logging: registra eventos de segurança em JSON

Clientes Supabase



Banco de Dados

Diagrama de Tabelas



Cognitive Engine

O Cognitive Engine é uma biblioteca pura de algoritmos TypeScript em `src/engine/`. Nenhum módulo importa do Supabase ou do Next.js — são funções testáveis em isolamento.

API Pública (`src/engine/index.ts`)



Risco de Esquecimento



- 1. Filtrar flashcards com `nextReview <= agora`
- 2. Ordenar: `high != medium != low`
- 3. Desempate: `nextReview` crescente

Mastery Score (0–100)



Sistema de Revisão Espaçada

Fluxo Completo

```
1. Usuário abre /review
2. ReviewView chama buildReviewQueue(flashcards)
3. Cards due são apresentados um a um
4. Usuário avalia: 1 (blackout) !' 5 (perfeito)
5. calcSM2() recalcula EF, interval, nextReview
6. reviewService.recordReviewCycle() persiste no banco
7. AppContext dispara UPDATE_CONTENT_PROGRESS
8. Fila atualizada no estado local
```

Qualidades de Avaliação (escala 0-5)

Qualidade	Significado
0-2	Falha total / confusão — reinicia interval para 1

Sistema de Autenticação

Fluxo Magic Link



- 5 requisições por IP em 15 minutos para /auth/*
- Retorna HTTP 429 com Retry-After header
- Implementado em memória (edge-compatible)

Segurança

HTTP Security Headers (`next.config.ts`)

Header	Valor
Content-Security-Policy	`default-src 'self'` + exceções necessárias

Sanitização (`src/lib/security/sanitize.ts`)



- Banner de consentimento em src/components/lgpd/ConsentBanner.tsx
- localStorage para persistir preferências
- DELETE /api/user/delete remove dados de todas as tabelas + auth.admin.deleteUser

Progressive Web App (PWA)

Visão Geral

O NeuroLearn é instalável como PWA em Android, iOS (Safari) e desktop (Chrome/Edge). A implementação usa Service Worker nativo sem next-pwa para evitar conflito com o webpack do @sentry/nextjs.

Manifesto (`public/manifest.webmanifest`)

Recurso	Estratégia

<code>`/_next/static/`</code>	Cache First — ativos imutáveis com hash
-------------------------------	---



O SW usa skipWaiting() no install e clients.claim() no activate para ativação imediata.

Push Notifications

Infraestrutura

- Biblioteca: web-push (server-side, Node.js)
- Protocolo: Web Push Protocol com VAPID
- Tabela: push_subscriptions no Supabase (RLS: auth.uid() = user_id)

Tabela `push\_subscriptions`

```
CREATE TABLE push_subscriptions (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id UUID NOT NULL REFERENCES auth.users(id) ON DELETE CASCADE,  
  endpoint TEXT NOT NULL,  
  p256dh TEXT NOT NULL,  
  auth TEXT NOT NULL,  
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  UNIQUE (user_id, endpoint)  
);  
ALTER TABLE push_subscriptions ENABLE ROW LEVEL SECURITY;  
CREATE POLICY "users can manage own push subscriptions"  
  ON push_subscriptions FOR ALL  
  USING (auth.uid() = user_id) WITH CHECK (auth.uid() = user_id);
```

Rota	Método	Descrição
<code>/api/push/subscribe`</code>	POST	Salva subscription via upsert <code>`onConflict: user_id.endpoint`</code>

Fluxo de envio (cron diário)

1. Buscar flashcards com next_review <= hoje
2. Agrupar por user_id (contagem)
3. Buscar push_subscriptions dos usuários afetados
4. Enviar via webpush.sendNotification() para cada endpoint
5. Remover subscriptions com HTTP 410 (expiradas)

Variáveis de Ambiente

Variável	Exposição	Descrição
<code>`NEXT_PUBLIC_VAPID_PUBLIC_KEY`</code>	Pública (browser)	Chave VAPID pública — usada no <code>`pushManager.subscribe()`</code>

Hook `usePushNotifications`

```
// src/hooks/usePushNotifications.ts
export function usePushNotifications(): {
  permission: 'default' | 'granted' | 'denied' | 'unsupported'
  isSubscribed: boolean
  loading: boolean
  subscribe(): Promise<boolean>
  unsubscribe(): Promise<void>
}
```

Componentes

- **ServiceWorkerRegistrar** — Client Component que registra /sw.js uma única vez após mount.

Renderiza null.

- **PushNotificationPrompt** — Banner fixo (bottom-center) que aparece 3s após mount se: permissão não negada, não subscrito, não dispensado (localStorage nl_push_dismissed).

Ambos adicionados ao AppShell.tsx após <AnalyticsIdentifier />.

Geração de Ícones

Gera os 4 ícones PNG em public/icons/ usando apenas módulos nativos do Node.js (zlib, fs, path). Não é necessário rodar em CI — os ícones estão commitados.

Estratégia de Testes

Unitários (Vitest)

Configuração (vitest.config.ts):

- Ambiente padrão: node
- Arquivos DOM: // @vitest-environment jsdom no topo do arquivo
- globals: true para matchers do jest-dom
- plugins: [react()] para transpilação JSX
- setupFiles: ['src/test/setup.ts']

Padrão de mock para Supabase:

Projetos.

- SUPABASE_SERVICE_ROLE_KEY (somente Node.js, nunca browser)
- TEST_USER_EMAIL

Comandos de Teste

Métrica	Mínimo
Statements	80%

Estratégia de IA

Configuração

--

Prompts centralizados para:

- buildFlashcardPrompt — geração de flashcards a partir de conteúdo e highlights
- buildTeachingPrompt — análise da explicação do usuário no Modo Professor
- buildCoachPrompt — geração de recomendações personalizadas pelo Cognitive Coach
- buildQuizPrompt — geração de distratores plausíveis para o Quiz Adaptativo

Endpoints de IA

Rota	Entrada	Saída
<code>POST /api/ai/generate-flashcards`</code>	<code>`notes`, `highlights`, `title`, `count`</code>	Array de <code>{ front, back }</code>

Todos os schemas de validação estão centralizados em `src/lib/ai/validation.ts` (Zod).

Quiz Adaptativo (`/active`` — modo Auto-Avaliação)

```
1. Usuário seleciona modo "0>Yé Auto-Avaliação" e escolhe um conteúdo
2. loadQuiz() filtra flashcards do conteúdo, ordena por mastery (new != learning != review != strong),
   limita a 7
3. Promise.allSettled() chama /api/ai/generate-quiz em paralelo para cada card
4. Fisher-Yates embaralha [correct_answer + 3 distractors] para cada pergunta
5. Usuário responde != feedback imediato (verde/vermelho) != Próxima
6. Ao fim: placar + XP (10 por acerto) + lista de cards para revisar
```

Cards com mastery = 'new' aparecem primeiro para priorizar consolidação de pontos fracos.

Análise do Modo Professor (`/active` — modo Modo Professor)



Deploy e Variáveis de Ambiente

Variáveis obrigatórias (`.env.local` — nunca commitar)



Configuração Supabase (Dashboard)

- Site URL: `http://localhost:3003` (dev) / URL de produção
- Redirect URLs: `http://localhost:3003/auth/callback`
- Email Auth: habilitado
- Magic Link: habilitado

Build



Convenções de Código

TypeScript

- `strict: true` em `tsconfig.json`
- Sem `any` (use `unknown` + `type guards`)
- Interfaces para objetos de dados; `type` para unions e aliases
- Imports com alias `@/ != src/`

Nomenclatura

Tipo	Convenção	Exemplo
Componentes	PascalCase	`FormField.tsx`

Hooks

camelCase + `use` prefix

`useTheme.ts`

Commits (padrão descritivo)

- Apenas onde o "porquê" é não-óbvio
- Sem comentários que explicam "o quê" (o código já faz isso)
- Documentação em português

Performance

Frontend

- Server Components por padrão (zero bundle JS para lógica server)
- Lazy loading de Client Components com `dynamic()` quando adequado
- CSS Tailwind purged no build (apenas classes usadas)
- Fontes via next/font (sem layout shift)

Banco de Dados

- Queries paralelas no AppContext mount (4 queries simultâneas via `Promise.all`)
- `getSession()` em vez de `getUser()` no layout (elimina round-trip de verificação)
- Índices nas colunas `user_id` e `nextReview` (colunas de filtro principal)

Cognitive Engine

- Algoritmos síncronos e puros (zero latência de I/O)
- `buildReviewQueue` roda em memória (sem queries adicionais)

Troubleshooting

Erro 500 "Cannot find module './NNN.js'"

Causa: Build realizado com servidor dev ativo — chunks stale em cache.

Solução:

Magic Link não chega / Rate limit

Causa: Free tier Supabase limita envios de email.

Solução: Aguardar ~1h ou configurar SMTP customizado no dashboard Supabase.

Testes E2E falhando por auth

Causa: tests/e2e/.auth/user.json expirado.

Solução: Deletar o arquivo e rodar npm run test:e2e novamente (global.setup.ts regenera).

`expect is not defined` no Vitest

Causa: globals: true ausente no vitest.config.ts ou setup.ts não importa jest-dom.

Solução: Verificar globals: true e import '@testing-library/jest-dom' em src/test/setup.ts.

IDE mostrando erros TypeScript após edição

Causa: Cache de diagnósticos do IDE (stale).

Solução: Verificar o estado real com npx tsc --noEmit.

Supabase RLS bloqueando query

Causa: Query usando cliente browser mas sem sessão ativa.

Solução: Usar cliente server ou verificar auth.uid() na policy.

Gate de Qualidade

Antes de qualquer entrega:



